

25COP506 Artificial Intelligence

Search algorithms

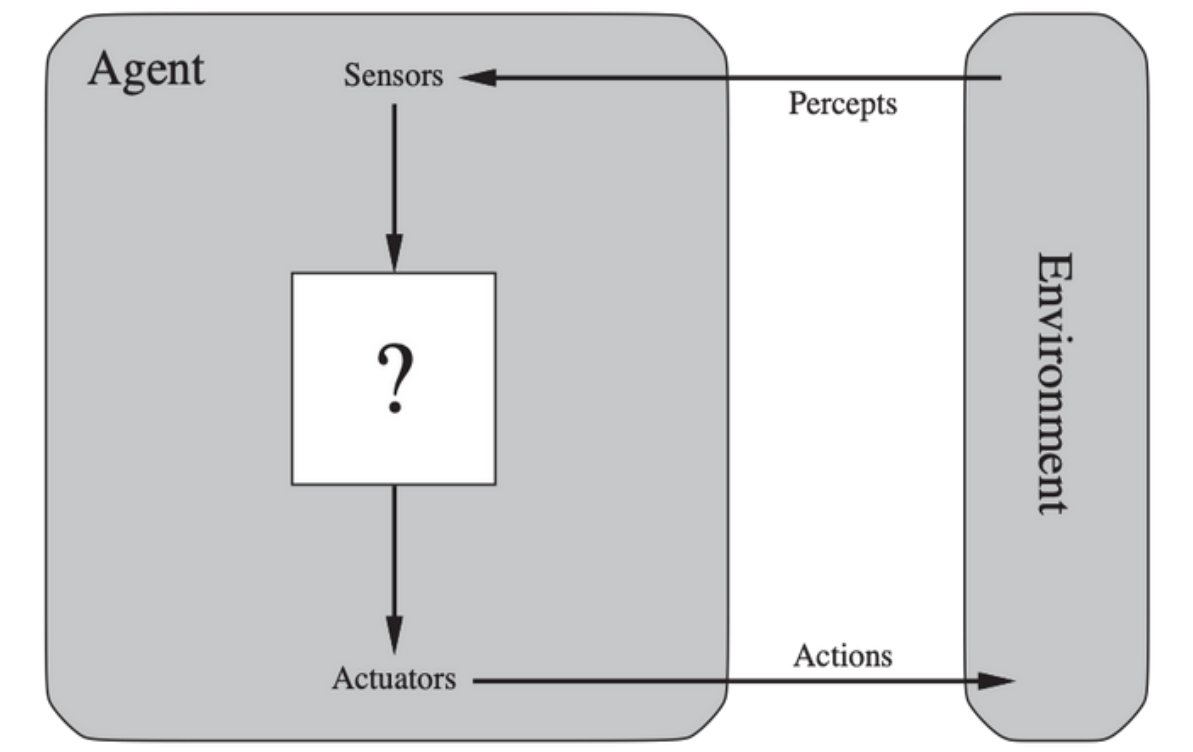
Yue Xie

Intelligent agents

- An agent perceives an environment and acts on it

- Properties of an environment:

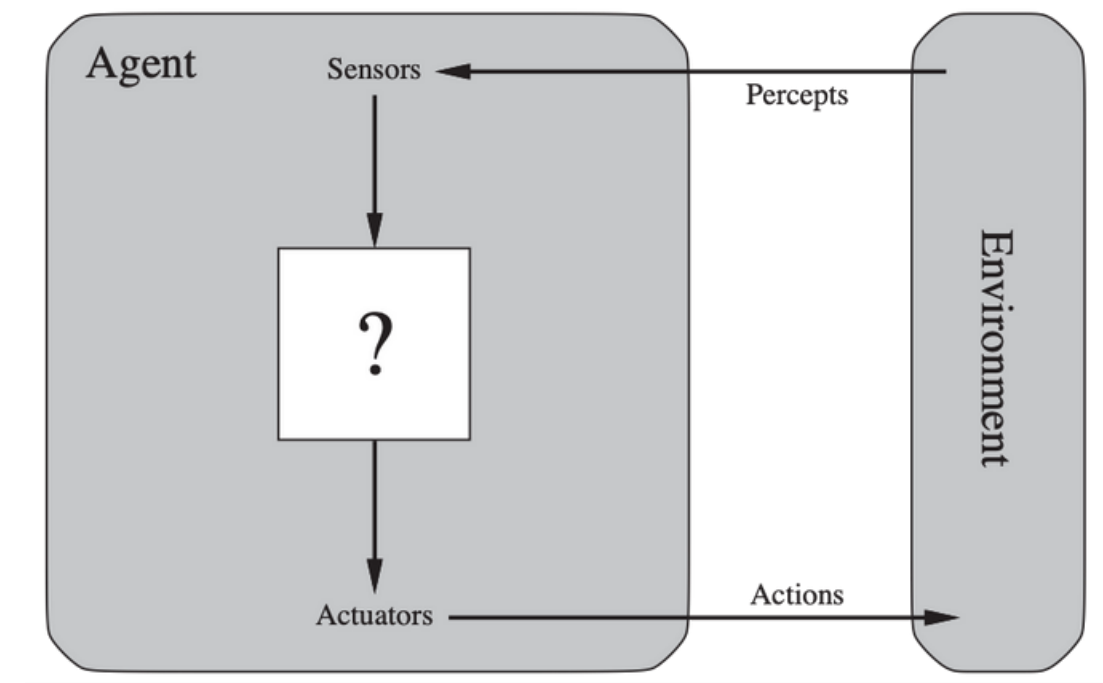
- Fully observable / partially observable
- Deterministic / stochastic
- Episodic / sequential
- Static / dynamics
- Discrete / continuous
-



- Discuss examples for each of the above

Intelligent agents

- Static agents versus learning agents.
- An agent **might be born with all the ‘instructions** or information about how to act in all situations.
- Alternatively, an agent might **be born with little knowledge** about the environment, but with a program that allows it to **learn from experience**
- **There are different** methods for learning, such as reward-based learning, unsupervised learning, imitation learning, supervised learning.



Intelligent agents

The concept of 'agent' is used in many areas of AI such as:

- Robotics Reinforcement learning Multi-agent systems (e.g. multi robot systems)
- Distributed systems (e.g. multiple entities across the internet)
- Artificial life systems and evolutionary computation

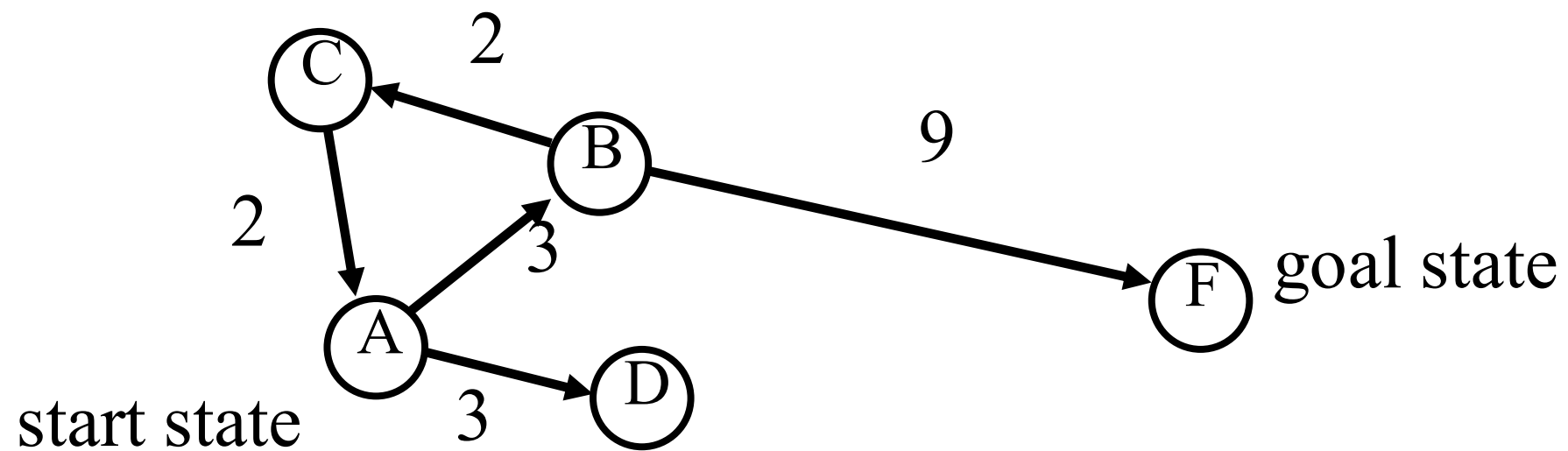
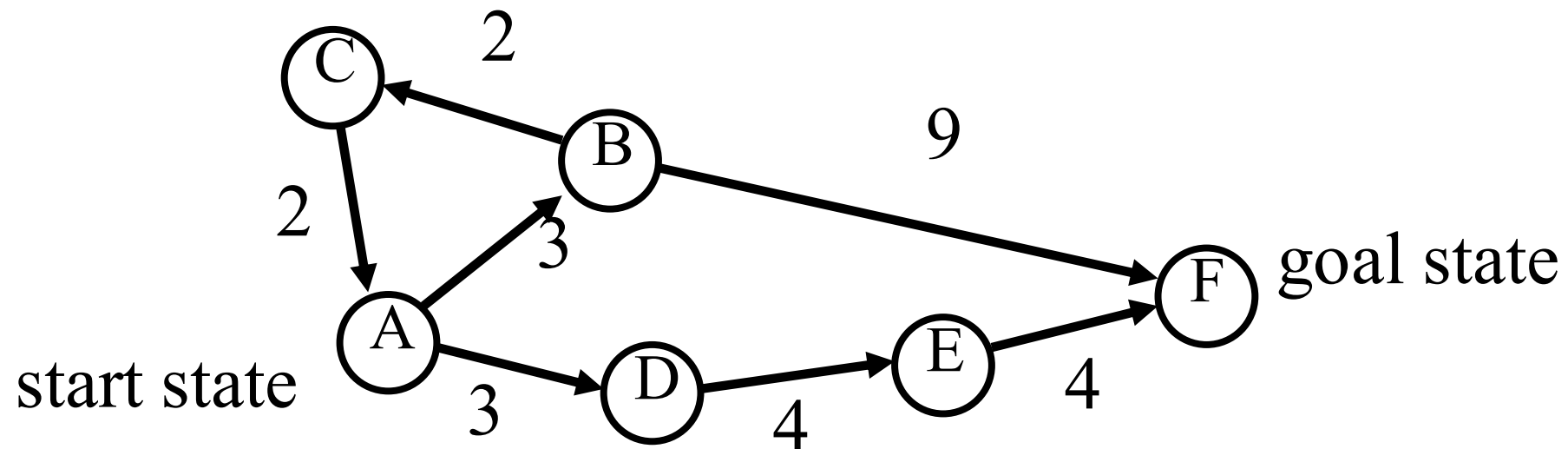
Areas of AI where the concept of agent is less used include:

- Deep learning
- Supervised and unsupervised learning
- Data mining/data science/statistics

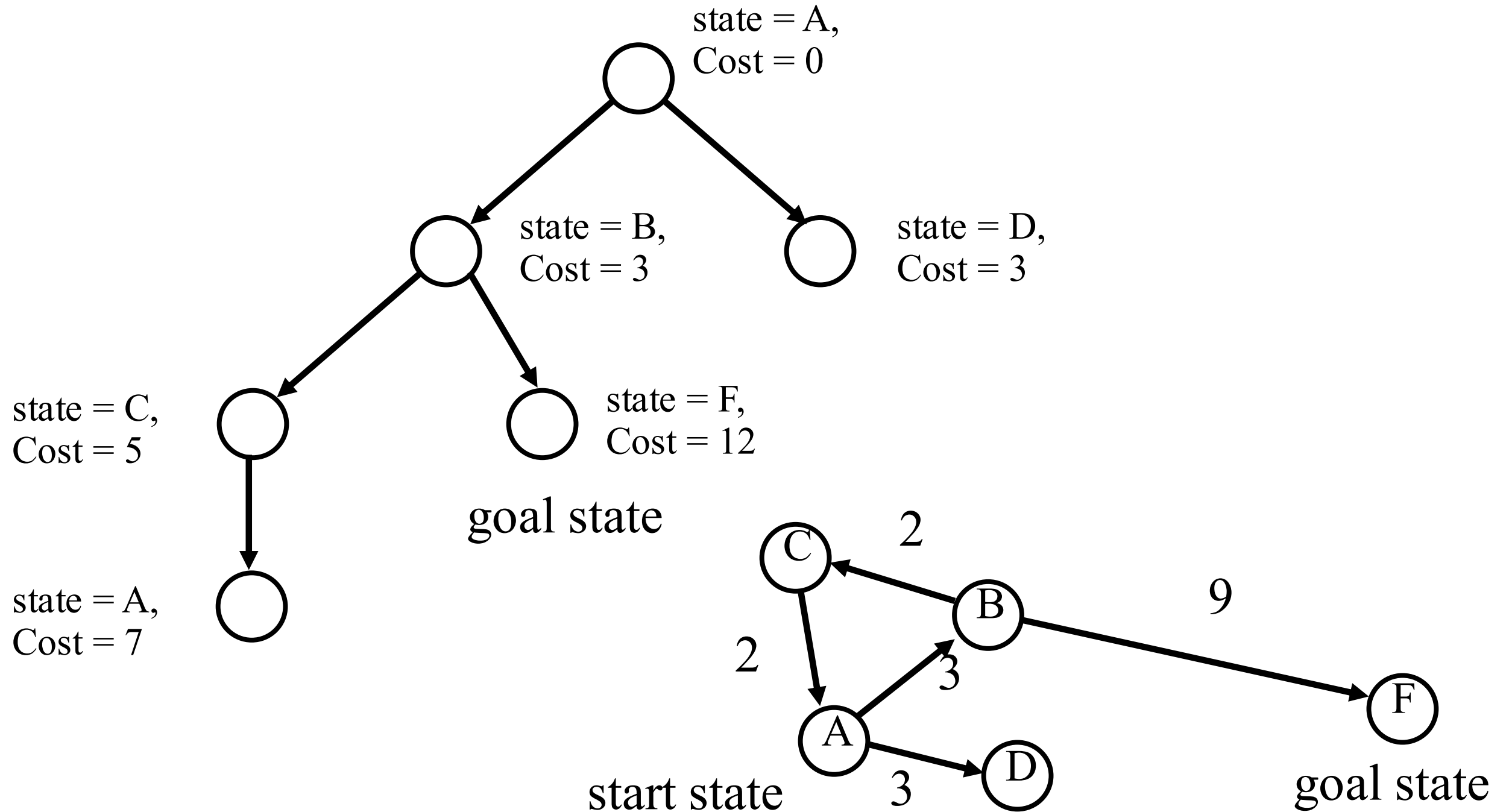
Motivations

- We have some actions that can change the state of the world
 - Change induced by an action is perfectly predictable
- Try to come up with a sequence of actions that will lead us to a **goal state**
 - May want to minimize number of actions
 - More generally, may want to minimize total cost of actions
- Do not need to execute actions in real life while searching for solution
 - Everything perfectly predictable anyway
- Search algorithms were studied considerably in GOFAI: they are essential algorithms in many real world problems, but became less important with the development of sub-symbolic AI and neural networks

A simple example: traveling on a graph

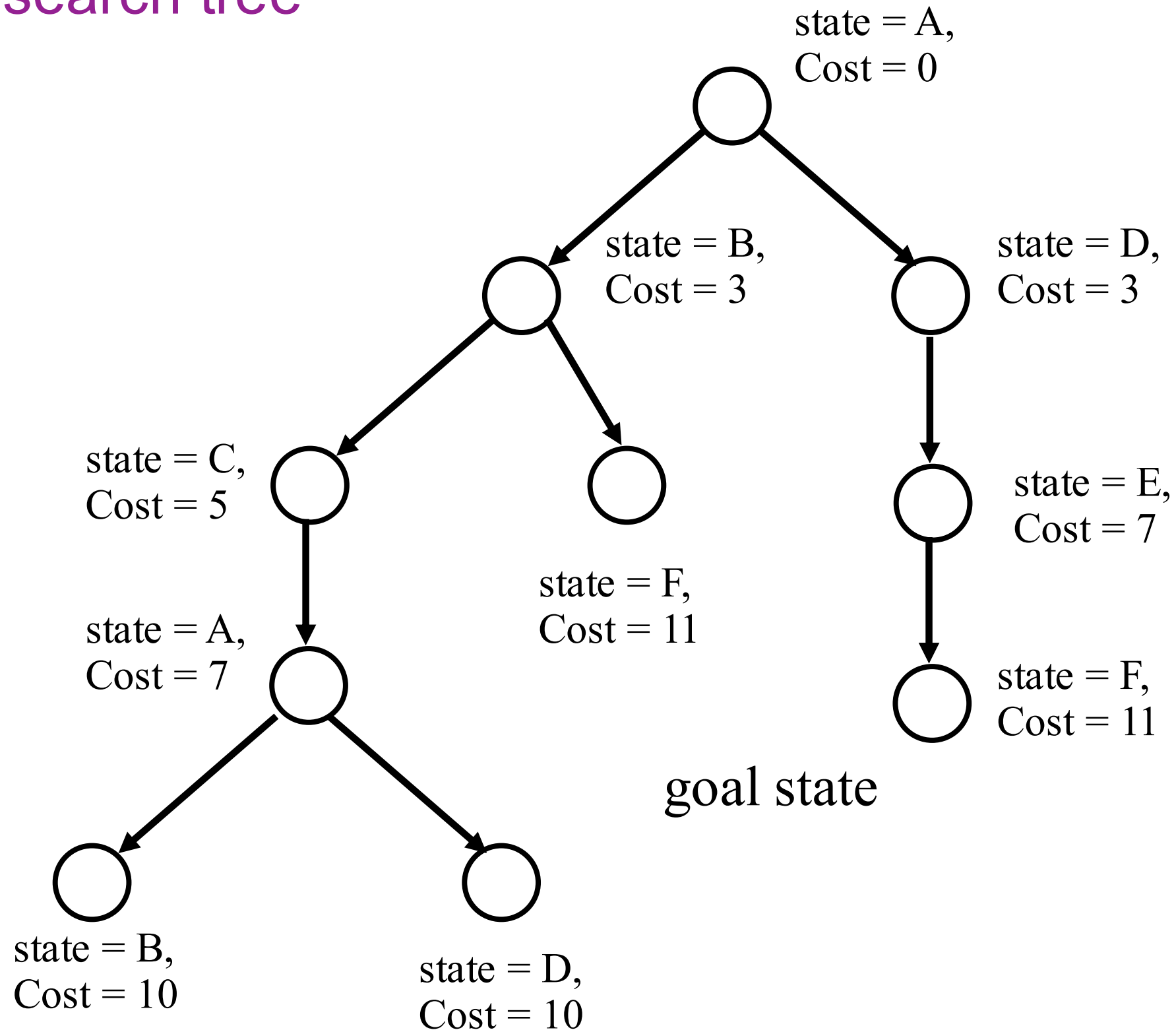


Search tree



search tree nodes and states are not the same thing!

Full search tree



Key concepts in search

- Set of states that we can be in
Including initial state and goal states (equivalently, a goal test)
- For every state, a set of actions that we can take
Each action results in a new state— typically defined by successor function
Given a state, produce all states that can be reached from it.
- Cost function that determines the cost of each action (or path = sequence of actions)
- Solution: path from initial state to a goal state
Optimal solution: solution with minimal cost

Example

- Example: the 8-puzzle
- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares
- **Initial state:** Any state can be designated as the initial state
- **Actions:** The simplest formulation defines the actions as movements of the blank space.
- **Transition model:** Given a state and action, this returns the resulting state
- **Goal test:** This checks whether the state matches the goal configuration
- **Path cost:** Each step costs

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

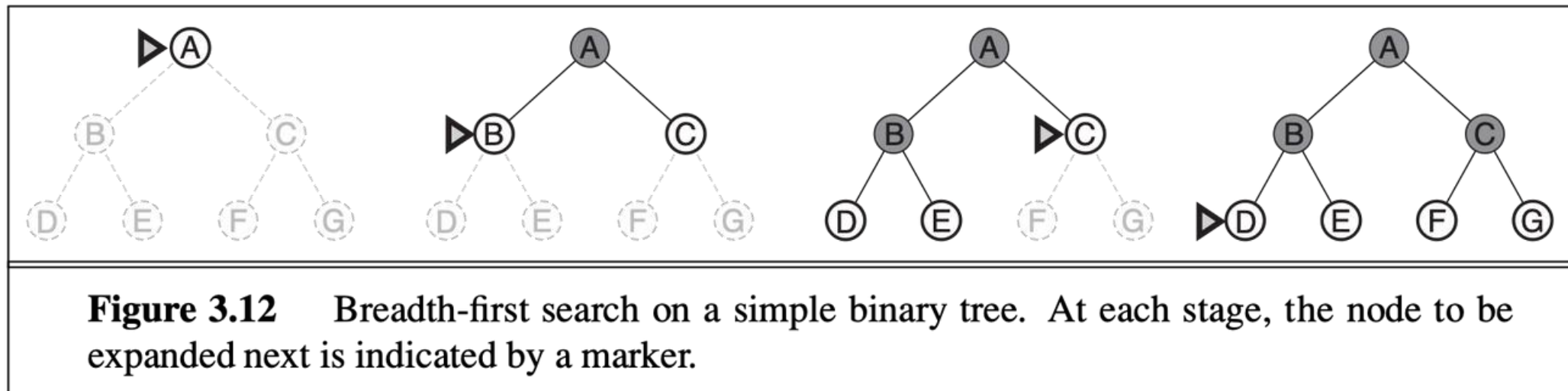
Genetic search algorithm

- Frontier = set of nodes generated but not expanded
- frontier := {node with initial state}
- Loop;
 - If frontier empty, declare failure
 - Choose and remove a node v from frontier
 - Check if v 's state s is a goal state: if so, declare success; if not, expand v , insert resulting nodes into frontier.
- Key question in search: which of the generated node we expand next?

Uninformed search strategies

- Given a state, we only know whether it is a goal state or not
- Can't say one non-goal state looks better than another non-goal state
- Can only traverse state space blindly in hope of somehow hitting a goal state at some point
 - Also called blind search
 - Blind does not imply unsystematic!

Breadth-first search: always expand the shallowest node



- Nodes are expanded in the same order in which they are generated

Frontier can be maintained as a First-in-First-out (FIFO) queue

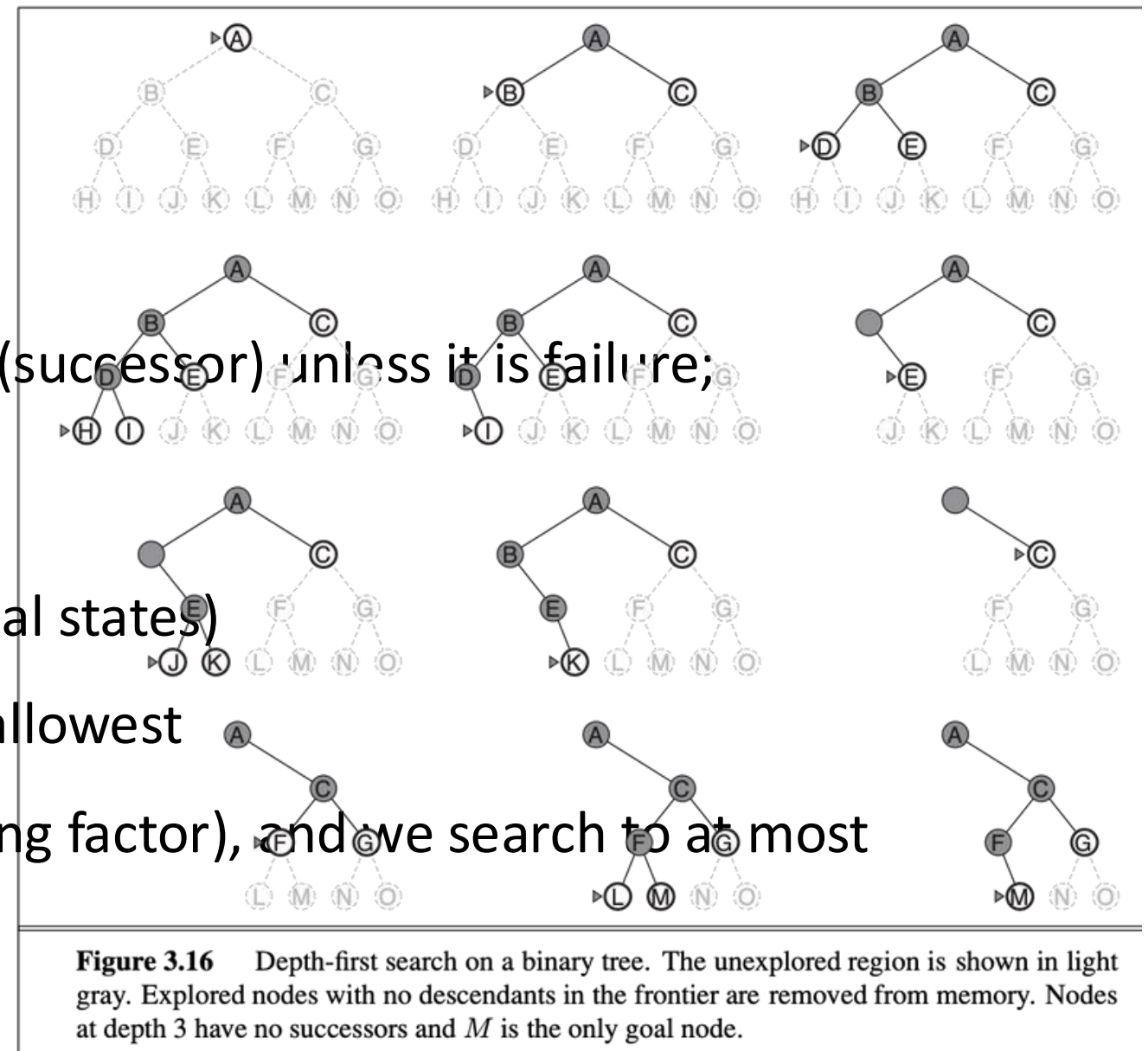
- BFS is complete: if a solution exist, one will be found
- BFS finds a shallowest solution

Not necessarily an optimal solution

- If every node has b successors (the branching factor), first solution is at depth d , then frontier size will at least b^d at some point (more time and space)

Depth-first search: Always expands the deepest node

- Frontier can be maintained as a Last-in-First-Out(LIFO) queue
- Also easy to implement recursively.
- DFS (node)
 - If goal (node) return solution (node);
 - For each successor of node :Return DFS(successor) unless it is failure;
 - Return failure
- Not complete (might cycle through non-goal states)
- If solution found, generally not optimal/shallowest
- If every node has b successors(the branching factor), and we search to at most depth m , frontier is at most bm .
 - Much better space requirement
 - Generally, don't even need to store all of frontier
- Time: still need to look at every node



Combining good properties of BFS and DFS

- Limited depth DFS: just like DFS, except never go deeper than some depth
- Iterative deepening DFS:
 - Call limited depth DFS with depth 0;
 - If unsuccessful, call with depth 1;
 - If unsuccessful, call with depth 2.
- Complete, finds shallowest solution
- Space requirements of DFS
- May seem wasteful timewise because replicating effort
 - Really not that wasteful because almost all effort at deepest level

Thinking about cost

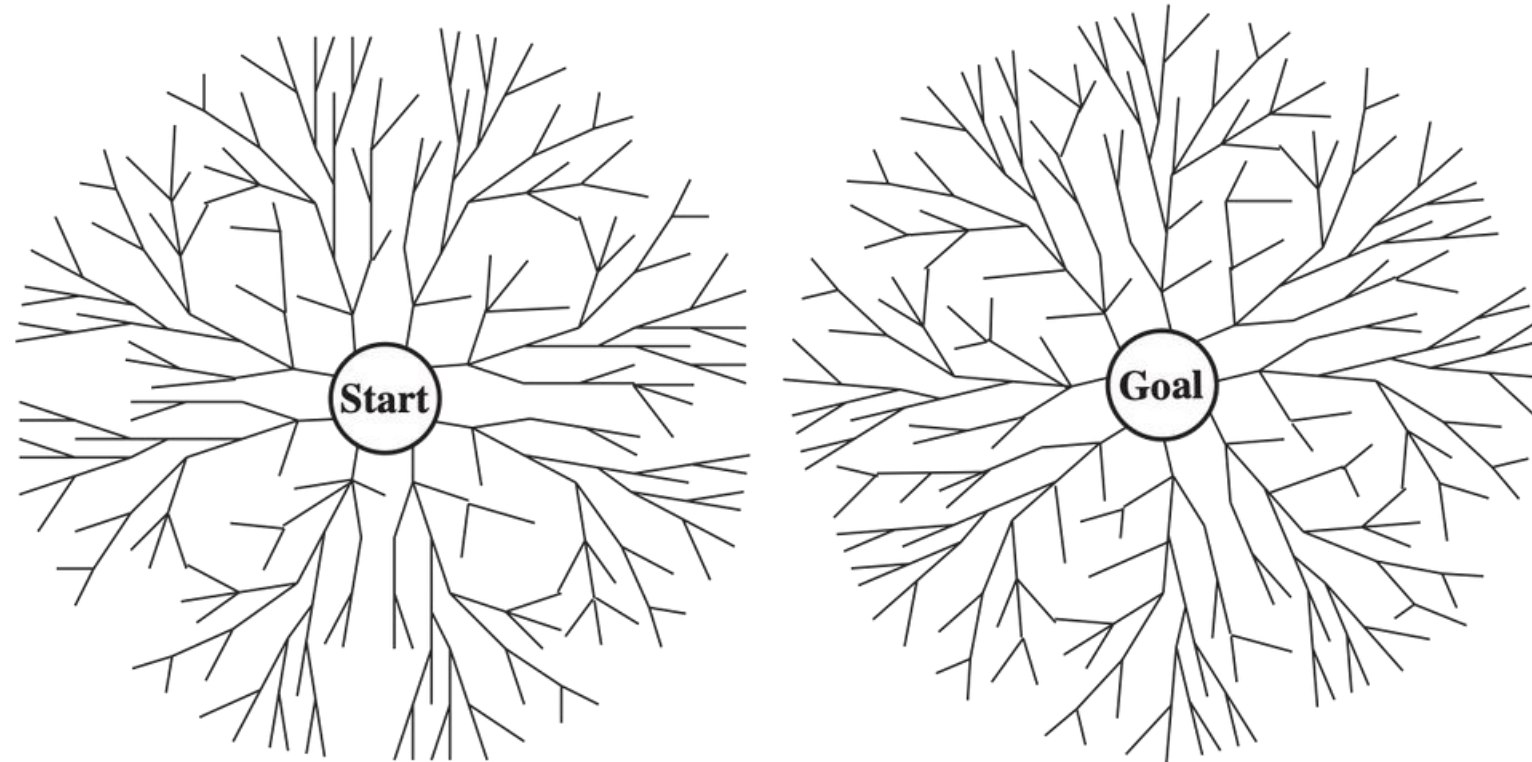
- BFS finds shallowest solution because always works on shallowest nodes first
- Similar idea: always on the lowest-cost node first (uniform-cost search)
- Will find optimal solution (assuming costs increase by at least constant amount along path)
- Will often pursue lots of short steps first
- If optimal cost is C , and cost increases by at least L each step, we can go to depth C/L

Searching backwards from the goal

- Sometimes can search backwards from the goal
 - Maze puzzles
 - Eight puzzle
 - Reaching location
 - What about the goal of “having visited all location?”
- Need to be able to compute predecessors instead of successors
- What’s the point?

Bidirectional search

- Start from the initial state and from the goal state, hoping that the two searches meet in the middle.



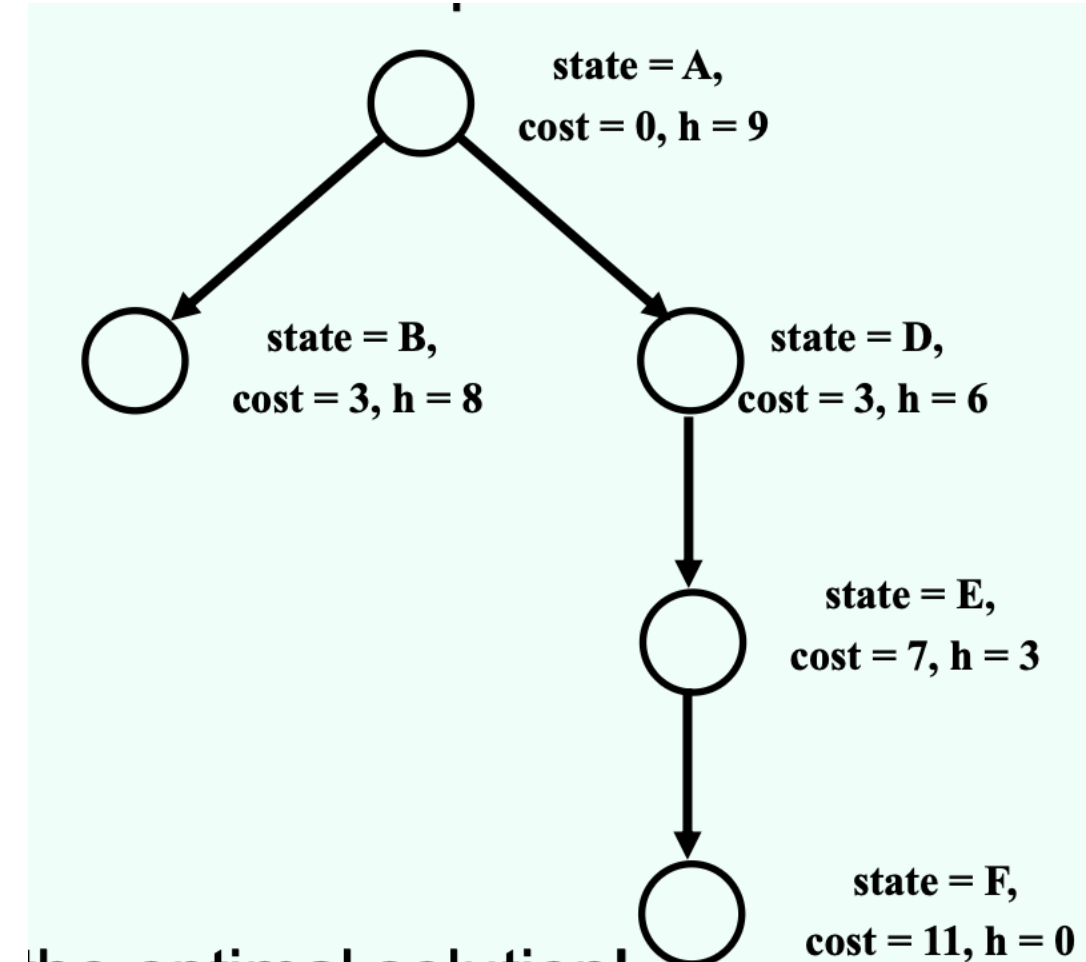
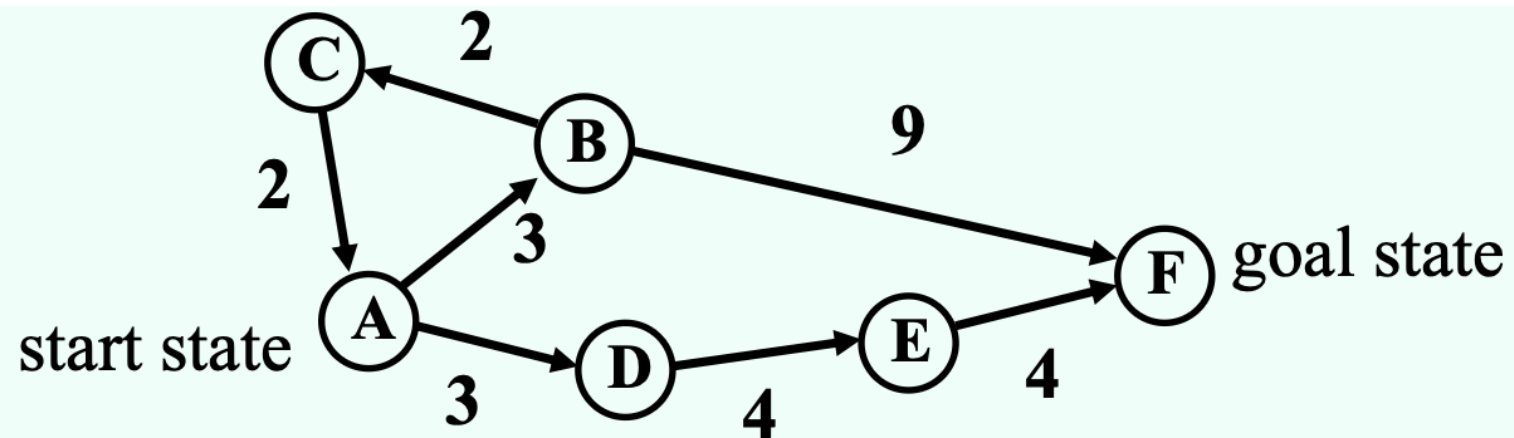
- Need to be able to figure out whether the frontier intersect (need to keep at least one frontier in memory..)
- Other than that, can do various kind of search on either tree, and get the corresponding optimality guarantees.

Informed (heuristic) searches

- So far, have assumed that no nongoal state looks better than another
- The best-first search is based on an **evaluation function** $f(n)$. This is the same as uniform-cost search, but instead of the cost, the evaluation function is used.
- A heuristic function $h(n)$ is used to estimate the cost from the current state to the goal
- In the greedy best-first search $f(n) = h(n)$

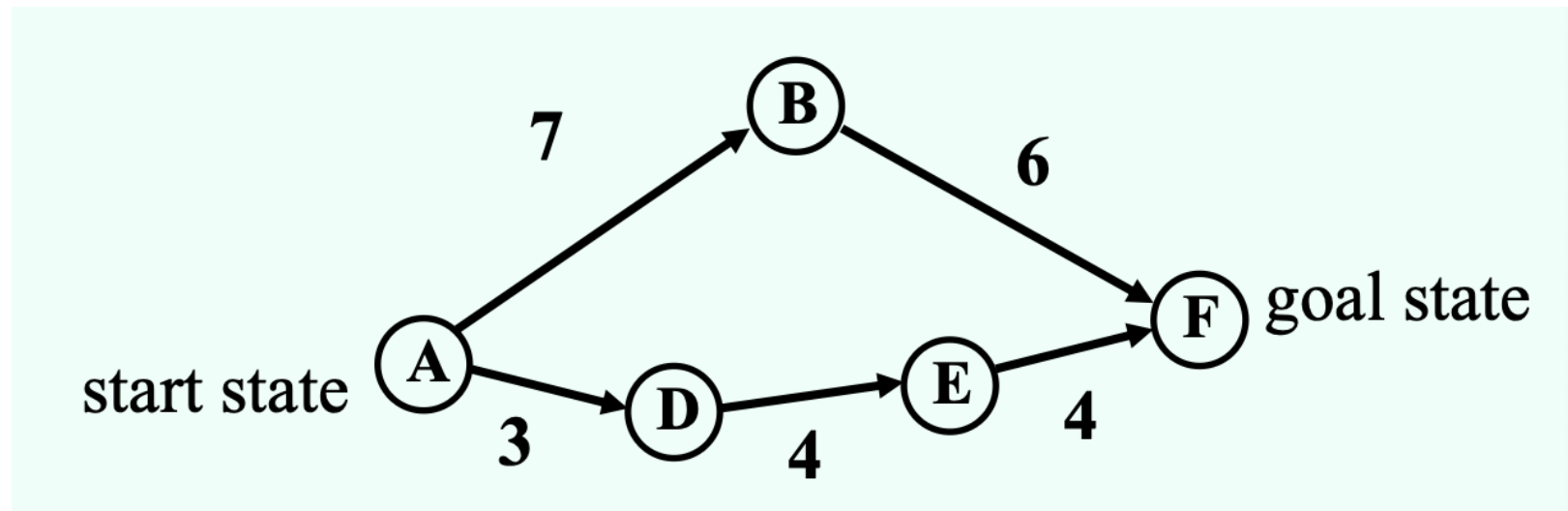
Greedy best-first search

- Greedy best-first search: expand nodes with lowest h values first



- Rapidly finds the optimal solution !
- Does it always?

A bad example for greedy



- Say: $h(A)=9$, $h(B)=5$, $h(D) = 6$, $h(E)=3$, $h(F)=0$
- Problem: greedy evaluates the promise of a node only by how far is left to go, does not take cost occurred already into account.

A* search

- Most widely know form of best-first search, pronounced A-star search
- It combines the cost to get to a node and the estimated cost to get to the goal
- Thus, $f(n) = g(n) + h(n)$

with $g(n)$ cost from initial state to current node and $h(h)$ estimated cost from current state to goal $f(n)$ estimated total cost from start to solution

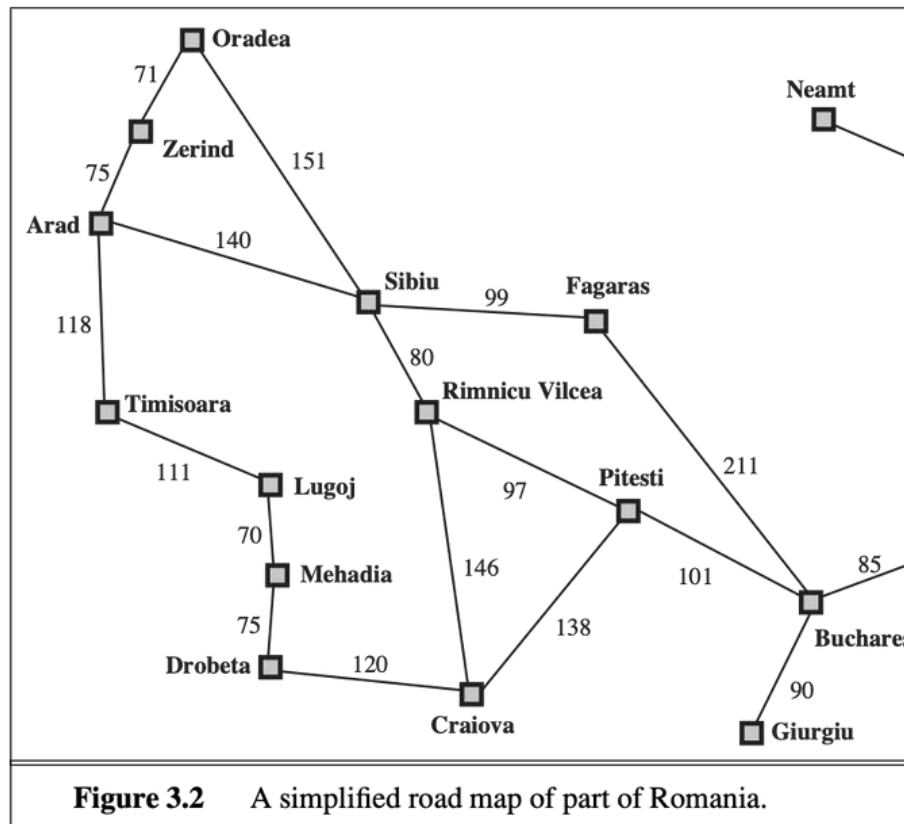
- Conditions for optimality:

$h(n)$ must be an admissible heuristic, i.e., it never overestimates the cost

Consistency (or monotonicity): the estimated cost cannot increase

- The tree-search version of A* is optimal if $h(n)$ is admissible, while the graph-search version is optimal if $h(n)$ is consistent.

cost : $g(n)$



heuristic function: $h(n)$

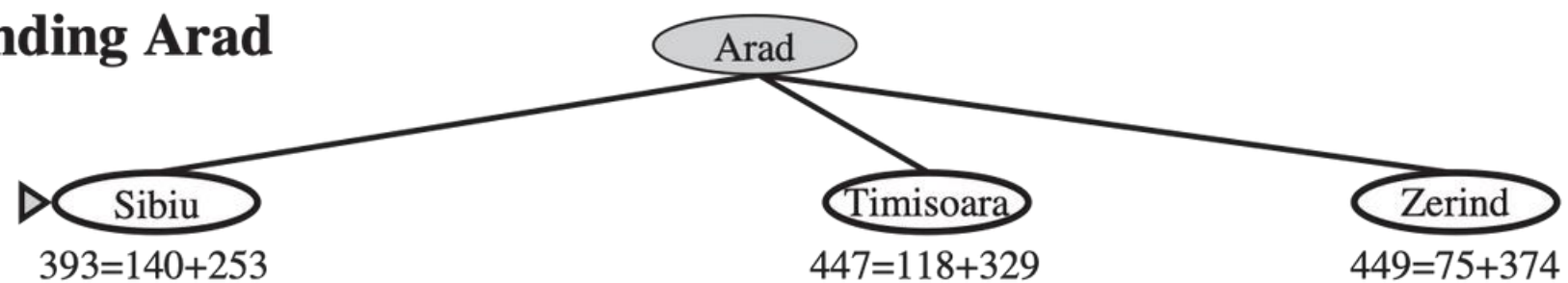
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

(a) The initial state



(b) After expanding Arad



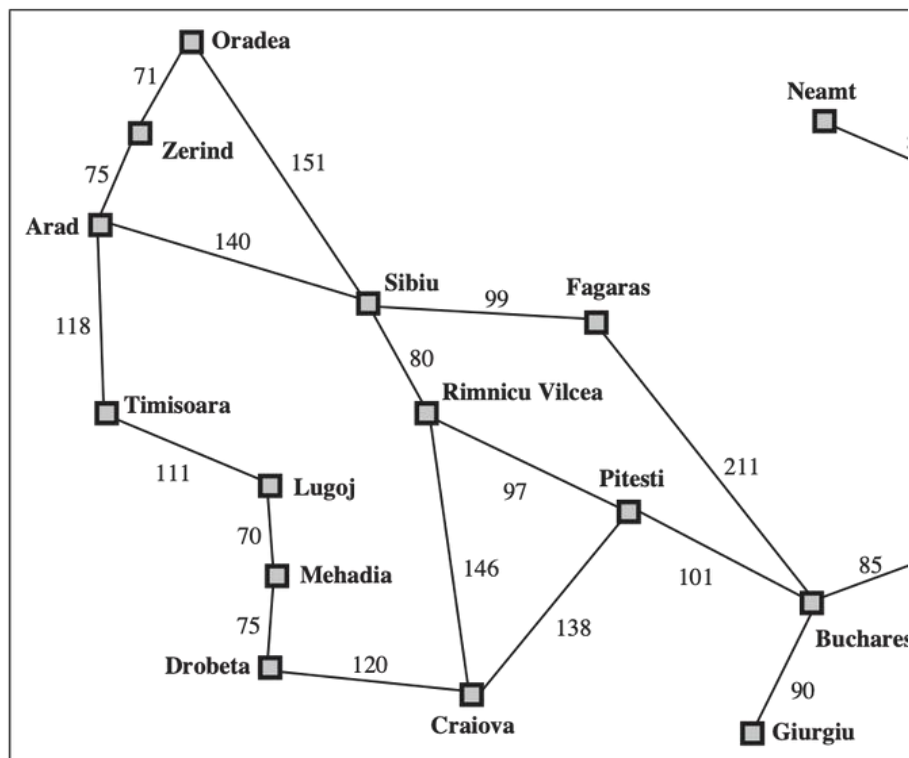
cost : $g(n)$ 

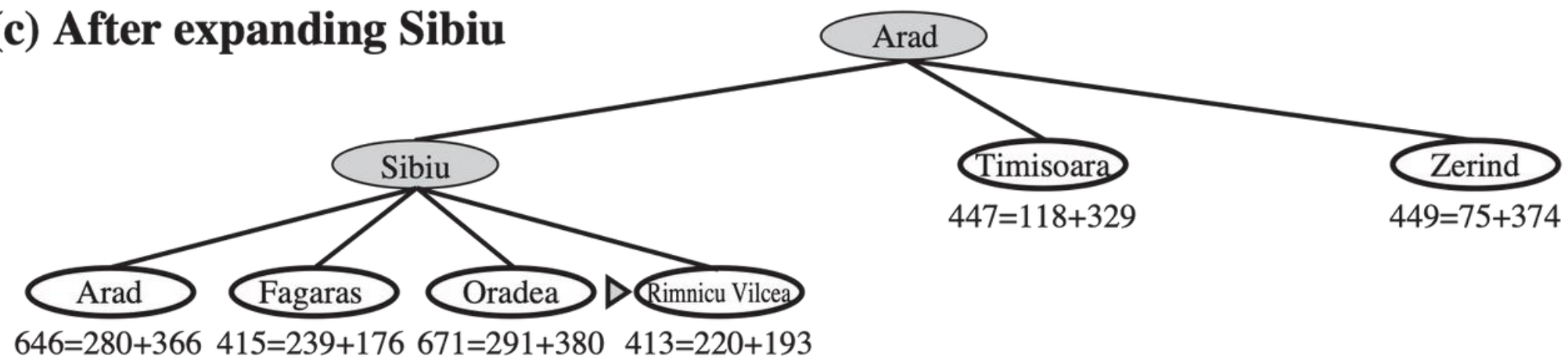
Figure 3.2 A simplified road map of part of Romania.

heuristic function: $h(n)$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

(c) After expanding Sibiu



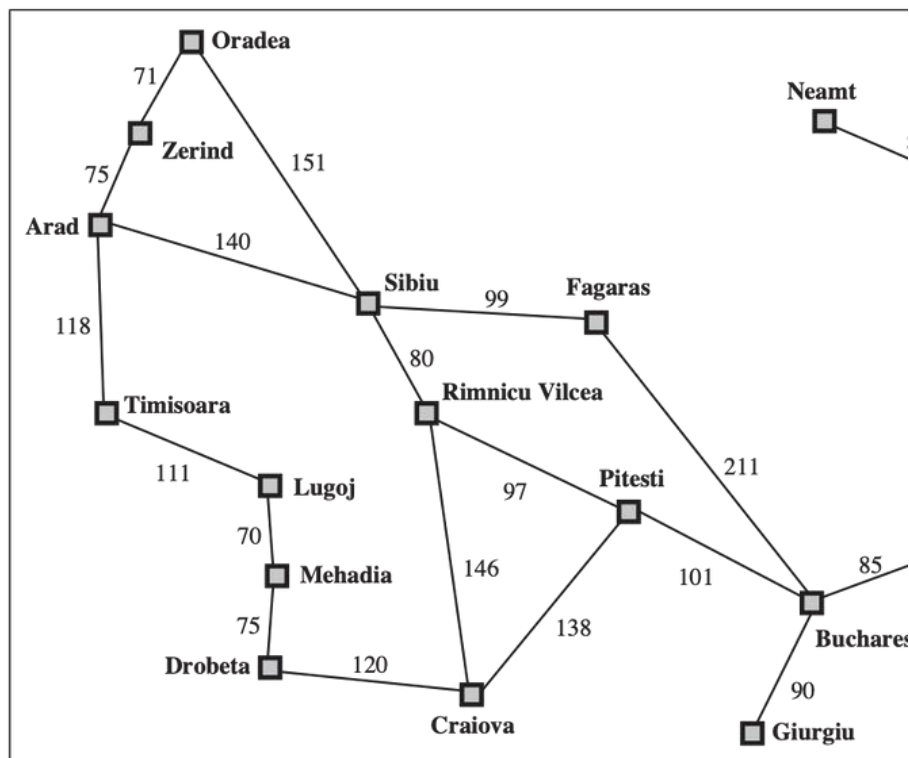
cost : $g(n)$ 

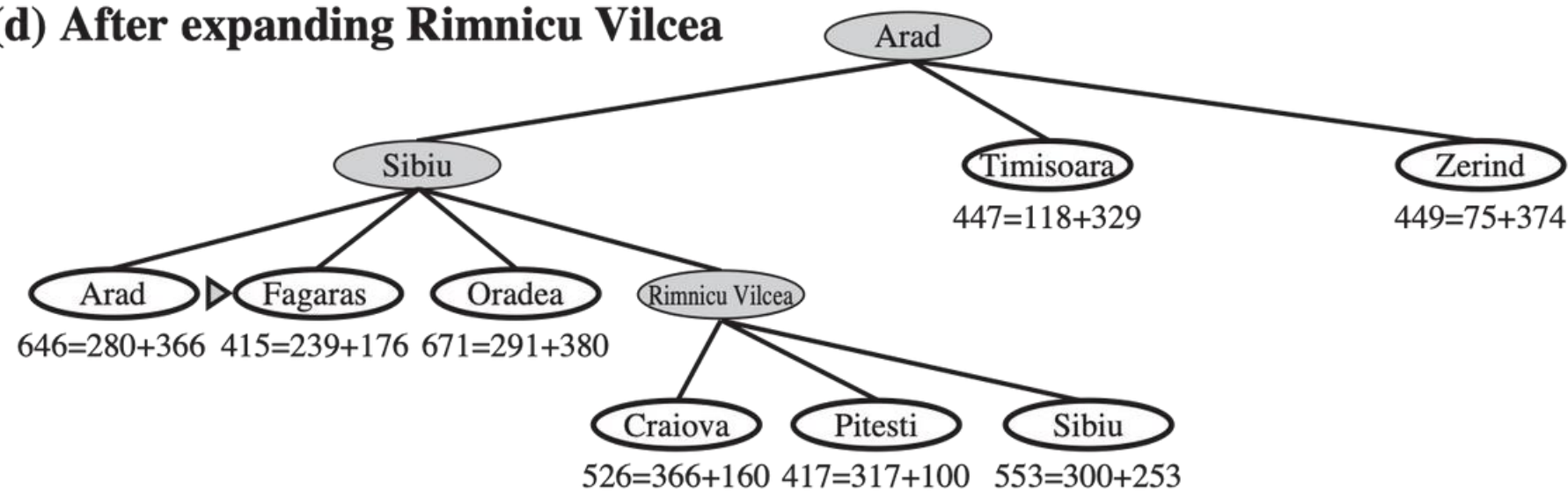
Figure 3.2 A simplified road map of part of Romania.

heuristic function: $h(n)$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

(d) After expanding Rimnicu Vilcea



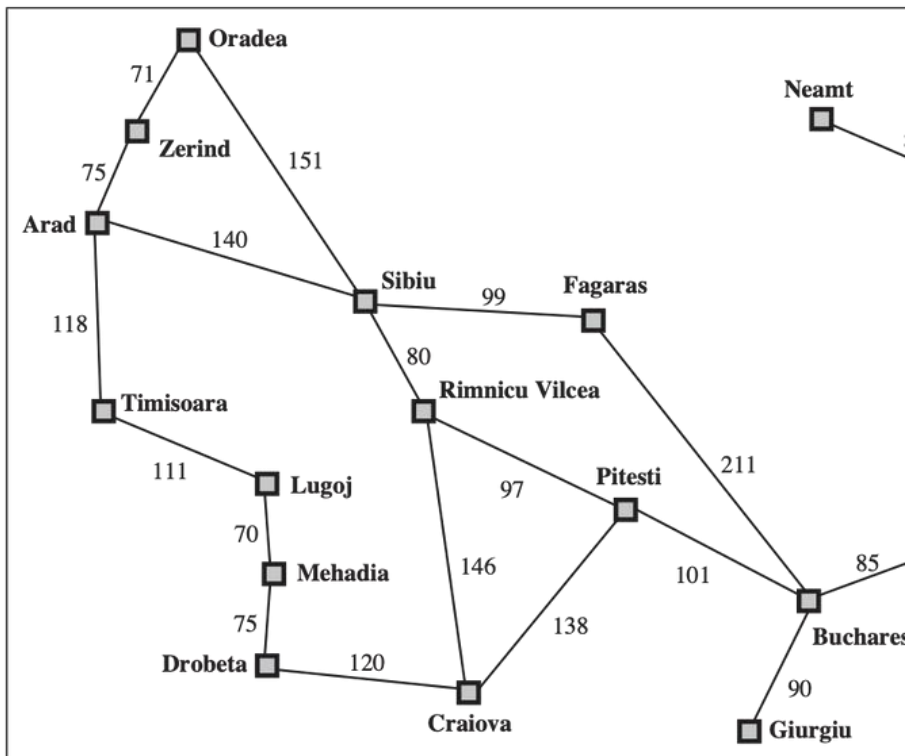
cost : $g(n)$ 

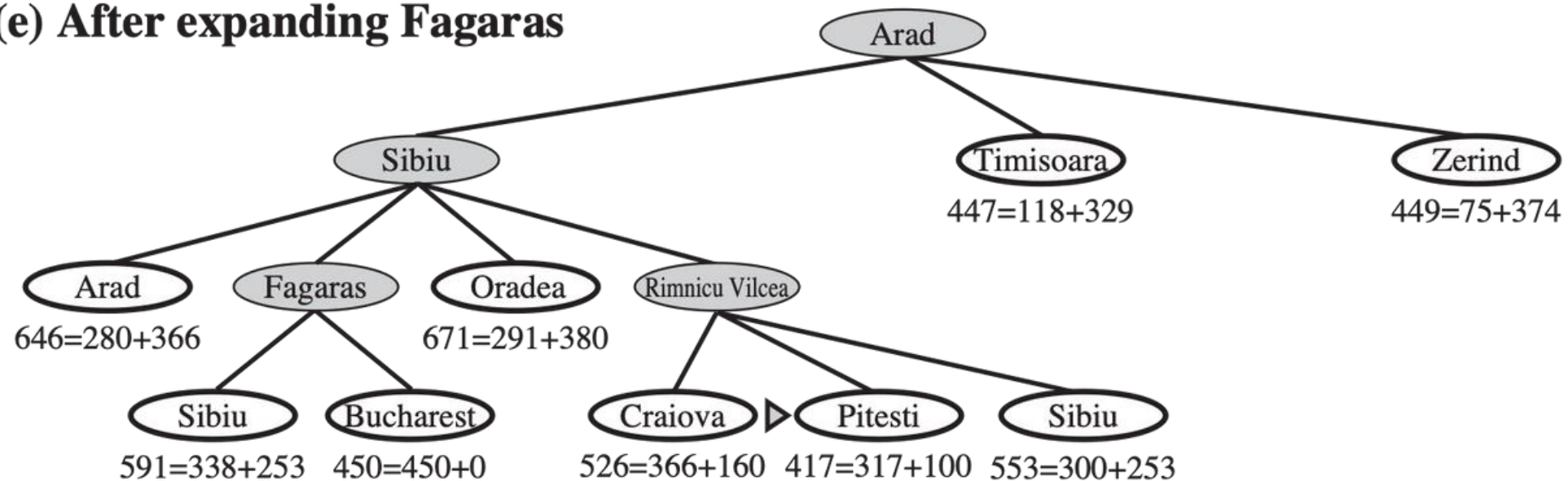
Figure 3.2 A simplified road map of part of Romania.

heuristic function: $h(n)$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

(e) After expanding Fagaras



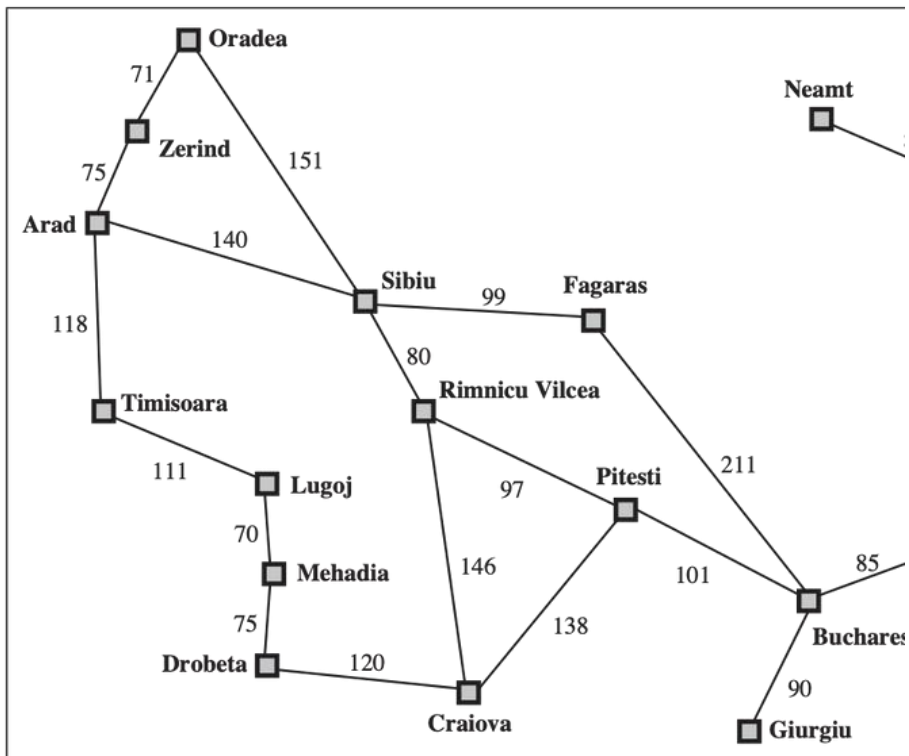
cost : $g(n)$ 

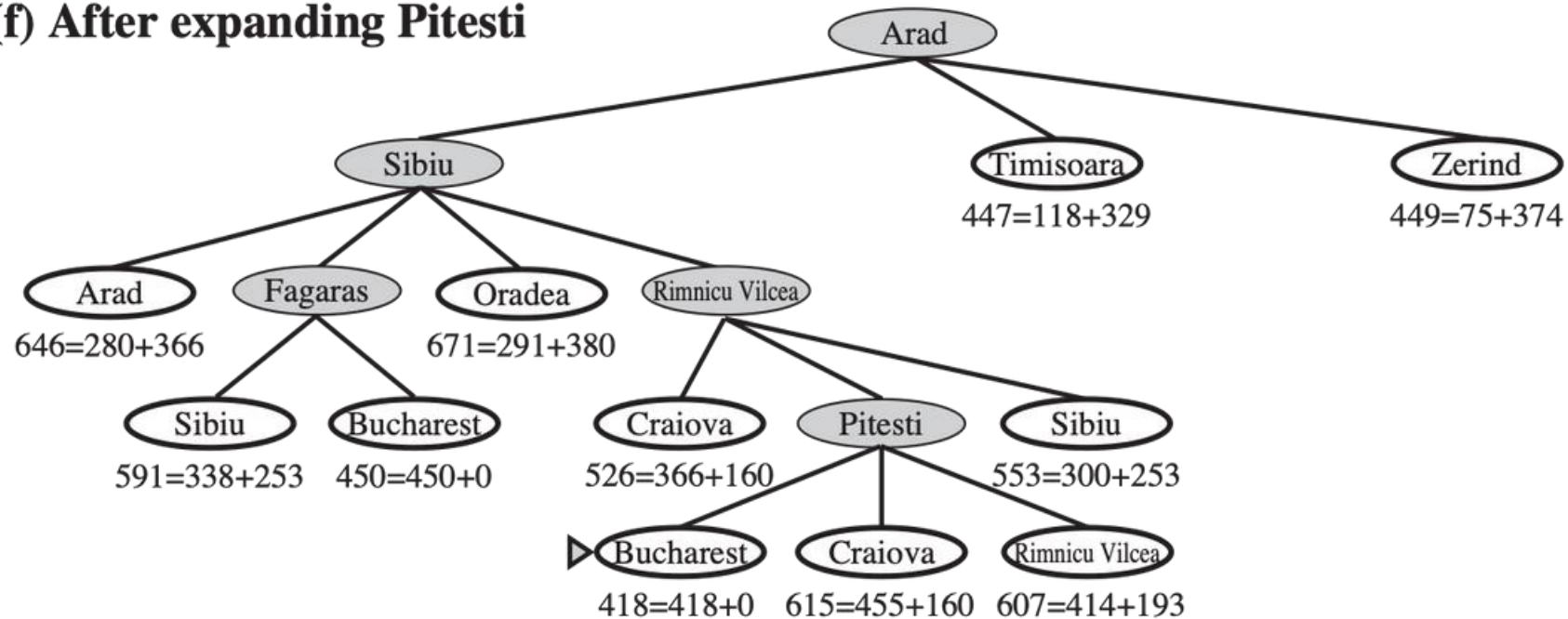
Figure 3.2 A simplified road map of part of Romania.

heuristic function: $h(n)$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

(f) After expanding Pitesti



Admissibility

- A heuristic is admissible if it never overestimates the distance to the goal
 - If n is the optimal solution reachable from n' , then $g(n) \geq g(n') + h(n')$
- Straight-line distance is admissible: can't hope for anything better than a straight road to the goal
- Admissible heuristic means that A^* is always optimistic

Consistency

- A heuristic is consistent if the following holds: if one step takes us from n' to n , then $h(n') \leq h(n) + \text{cost of step from } n' \text{ to } n$
 - Similar to triangle inequality
 - Equivalently, $g(n') + h(n') \leq g(n) + h(n)$
- Implies admissibility

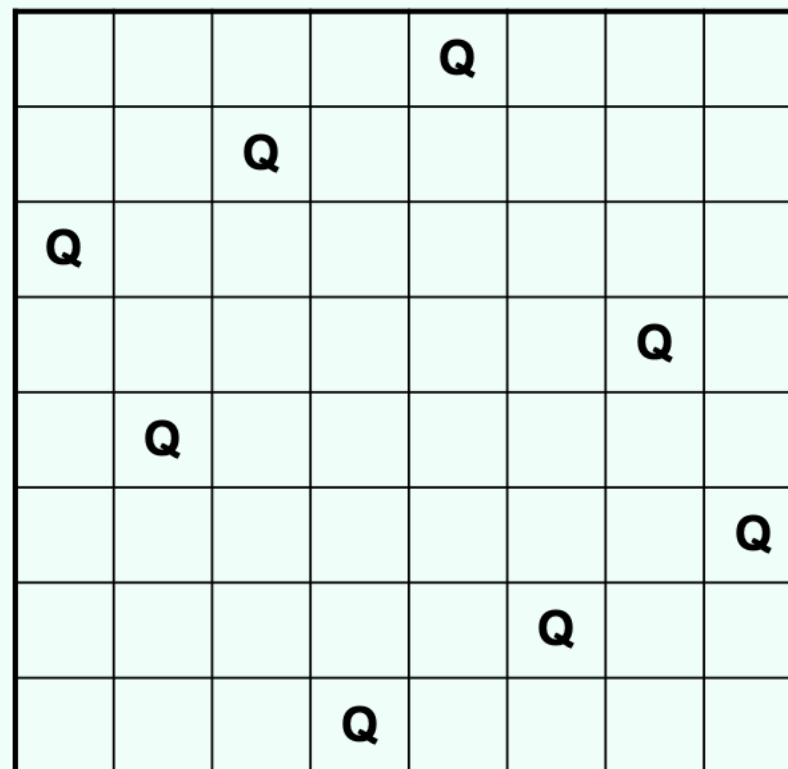
More details in Chapter 3 of Russell and Norvig (2016)

More search: when the path to the solution doesn't matter

- So far, looked at problems where the path was the solution
 - Traveling on a graph
 - Eight puzzle
- However, in many problems, we just want to find a goal state
 - Doesn't matter how we get there

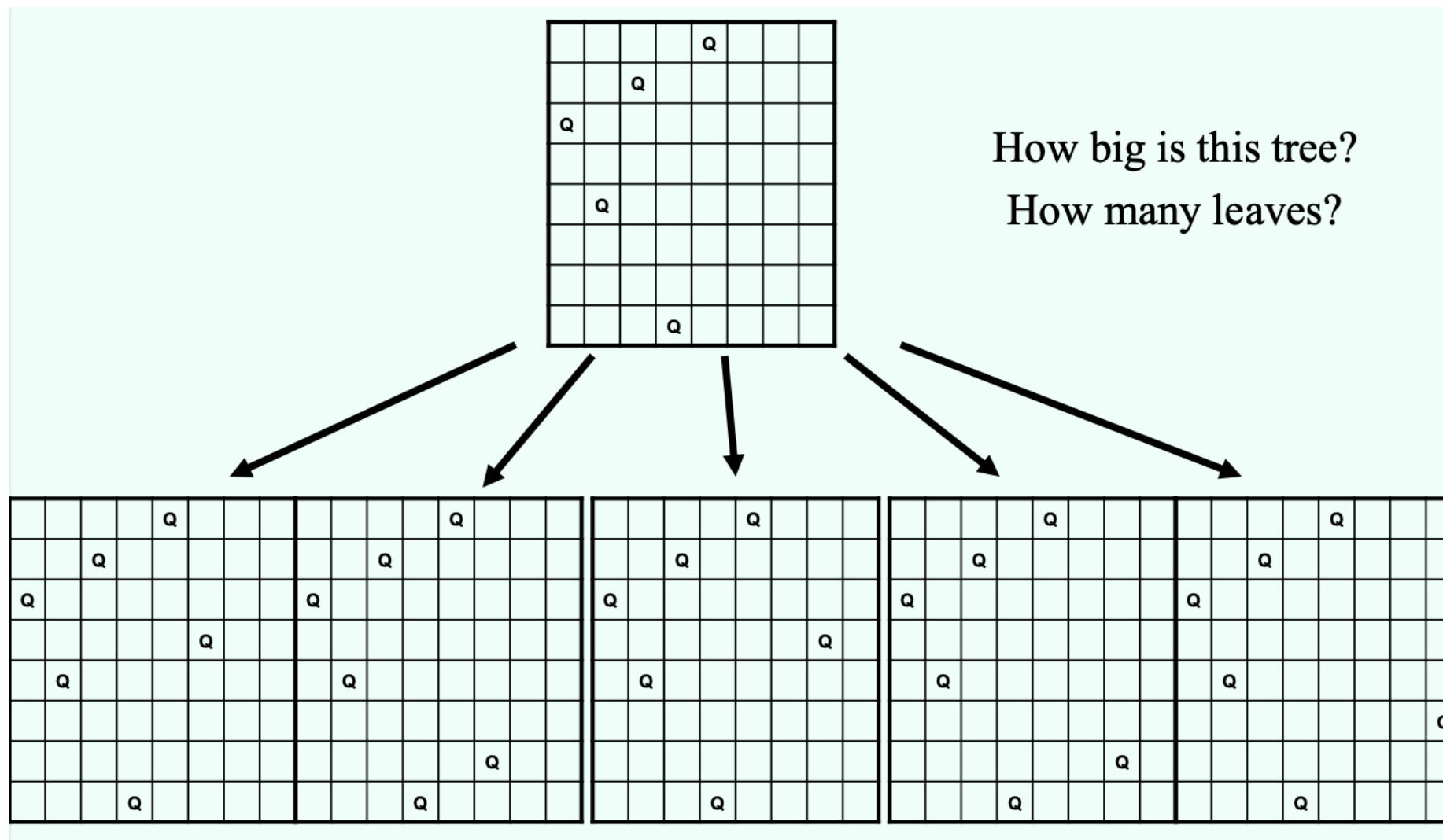
Queens puzzle

- Place eight queens on a chessboard so that no two attack each other



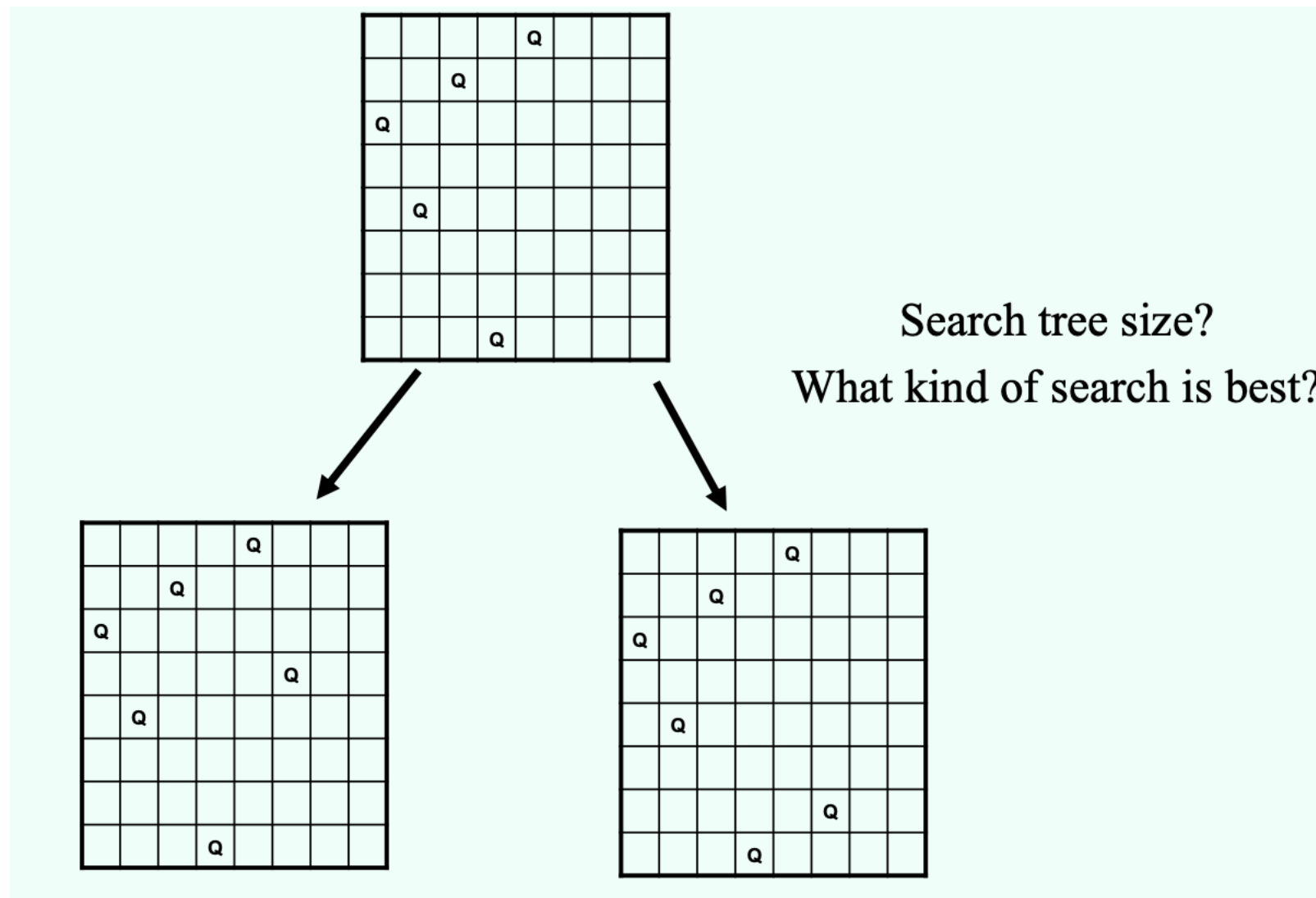
Search formulation of the queens puzzle

- Successors: all valid ways of placing additional queen on the board;
- Goal: eight queens places



Search formulation of the queens puzzle

- Successors: all valid ways of placing additional queen in the next column
- Goal: eight queens places



A different approach: optimization

- Let's say every way of placing 8 queens on a board, one per column, is feasible
- Now we introduce an objective: minimize the number of pairs of queens that attack each other
 - More generally, minimize the number of violated constraints
- Pure optimization

Local search: hill climbing

- Start with a complete state
- Move to successor with best (or at least better) objective value
 - Successor: move one queen within its column
- Local search can get stuck in a local optimum

How to avoiding getting stuck with local search